

GUIDE GIT

Du fondamental à la pratique

Concepts • Commandes • Branches • Dépôt Distant • TP Pratique

Auteur : papalioune03@gmail.com

Mars 2026

Table des Matières

1. Pourquoi Git ?

Dans le développement logiciel moderne, gérer les modifications du code source est un défi fondamental. Git répond à des problèmes concrets que tout développeur rencontre tôt ou tard.

1.1 Les Problèmes Sans Contrôle de Version

- Impossibilité de revenir à une version précédente en cas de bug
- Travail en équipe difficile : qui a modifié quoi, et quand ?
- Perte de code en cas d'écrasement accidentel d'un fichier
- Gestion manuelle des sauvegardes (projet_v1, projet_v2, projet_final...)
- Aucune traçabilité des changements effectués

1.2 Les Avantages de Git

- Historique complet de toutes les modifications
- Retour en arrière vers n'importe quel état antérieur
- Collaboration fluide entre plusieurs développeurs
- Travail en parallèle via les branches
- Traçabilité : chaque modification est associée à un auteur et une date
- Sauvegarde distribuée : chaque clone est une sauvegarde complète
- Fusion intelligente des modifications concurrentes

1.3 Git dans l'Écosystème Professionnel

Git est aujourd'hui le standard industriel du contrôle de version. Il est utilisé par des millions de développeurs et d'entreprises à travers le monde, des startups aux GAFAM. Des plateformes comme GitHub, GitLab et Bitbucket sont entièrement construites autour de Git.

2. Qu'est-ce que Git ?

2.1 Définition

Git est un système de contrôle de version distribué (DVCS - Distributed Version Control System) créé par Linus Torvalds en 2005 pour gérer le développement du noyau Linux. Il permet de suivre les modifications apportées à des fichiers au fil du temps et de coordonner le travail entre plusieurs personnes.

2.2 Caractéristiques Clés

- Distribué : chaque développeur possède une copie complète de l'historique
- Rapide : la plupart des opérations s'effectuent localement
- Intégrité : chaque commit est identifié par un hash SHA-1 unique
- Non-linéaire : supporte des milliers de branches parallèles
- Open Source : libre et gratuit

2.3 Les Trois Zones de Git

Git organise le travail en trois zones distinctes :

Commande	Description
<code>Working Directory</code>	Zone de travail locale : vos fichiers modifiés mais non encore ajoutés à Git
<code>Staging Area (Index)</code>	Zone de préparation : fichiers ajoutés avec 'git add', prêts à être commités
<code>Repository (.git)</code>	Base de données Git locale : contient tout l'historique des commits

2.4 Le Cycle de Vie d'un Fichier

- Untracked : nouveau fichier, non suivi par Git
- Staged : ajouté à la zone de préparation (git add)
- Committed : sauvegardé dans le dépôt local (git commit)
- Modified : fichier suivi mais modifié depuis le dernier commit

3. Installation

3.1 Windows

Télécharger Git for Windows depuis le site officiel : <https://git-scm.com/download/win>
L'installateur fournit Git Bash, un terminal compatible Unix pour utiliser Git sous Windows.

3.2 macOS

Via Homebrew (recommandé) :

```
brew install git
```

Ou télécharger depuis : <https://git-scm.com/download/mac>

3.3 Linux (Ubuntu/Debian)

```
sudo apt update  
sudo apt install git
```

3.4 Vérification de l'installation

```
git --version
```

Résultat attendu : git version 2.x.x

4. Configuration

Avant d'utiliser Git, il est essentiel de configurer votre identité. Cette information est attachée à chaque commit que vous créez.

4.1 Configuration Globale

```
# Définir votre nom
git config --global user.name "Votre Nom"

# Définir votre email
git config --global user.email "votre.email@exemple.com"

# Définir l'éditeur par défaut (ex: VS Code)
git config --global core.editor "code --wait"

# Définir la branche par défaut
git config --global init.defaultBranch main
```

4.2 Niveaux de Configuration

Commande	Description
<code>--system</code>	S'applique à tous les utilisateurs du système (/etc/gitconfig)
<code>--global</code>	S'applique à l'utilisateur courant (~/.gitconfig)
<code>--local</code>	S'applique uniquement au dépôt courant (.git/config)

4.3 Vérification de la Configuration

```
# Voir toute la configuration
git config --list

# Voir une valeur spécifique
git config user.name
```

4.4 Le Fichier .gitignore

Le fichier .gitignore permet d'exclure certains fichiers du suivi Git (fichiers temporaires, secrets, dépendances...).

```
# Exemple de .gitignore
node_modules/
*.log
.env
__pycache__/.
.DS_Store
```

5. Commandes de Base

5.1 Initialisation et Clonage

Commande	Description
<code>git init</code>	Initialise un nouveau dépôt Git dans le répertoire courant
<code>git clone <url></code>	Clone un dépôt distant en local
<code>git clone <url> <dossier></code>	Clone dans un dossier spécifique

5.2 Vérification du Statut

Commande	Description
<code>git status</code>	Affiche l'état complet des fichiers (modifiés, staged, untracked)
<code>git status -s</code>	Version courte : A=staged, M=modifié, ??=non suivi

```
$ git status -s
A nouveau_fichier.txt      # Staged (ajouté)
M fichier_modifie.txt      # Modified (non staged)
M fichier_stage.txt        # Modified + staged
?? fichier_inconnu.txt     # Untracked
```

5.3 Ajout à la Zone de Staging

Commande	Description
<code>git add <fichier></code>	Ajoute un fichier spécifique à la zone de staging
<code>git add .</code>	Ajoute tous les fichiers modifiés du répertoire courant
<code>git add -p</code>	Ajoute interactivement par portions (hunks)
<code>git add -A</code>	Ajoute tous les fichiers (nouvelles créations et suppressions)

5.4 Commit

Commande	Description
<code>git commit -m 'message'</code>	Crée un commit avec un message court
<code>git commit -am 'message'</code>	Ajoute et commit en une seule commande (fichiers suivis)
<code>git commit --amend</code>	Modifie le dernier commit (message ou contenu)
<code>git commit --amend --no-edit</code>	Modifie le contenu sans changer le message

Bonnes pratiques pour les messages de commit :

- Utiliser le mode impératif : 'Ajoute', 'Corrige', 'Supprime'
- Limiter la première ligne à 50 caractères

- Décrire le POURQUOI, pas le COMMENT

5.5 Visualisation des Différences

Commande	Description
<code>git diff</code>	Affiche les modifications non encore staged
<code>git diff --staged</code>	Affiche les modifications staged (prêtes à commiter)
<code>git diff <branche1> <branche2></code>	Compare deux branches
<code>git diff <commit1> <commit2></code>	Compare deux commits

5.6 Historique des Commits

Commande	Description
<code>git log</code>	Affiche l'historique complet des commits
<code>git log --oneline</code>	Historique compact (une ligne par commit)
<code>git log --graph --oneline</code>	Historique avec arbre des branches
<code>git log -n 5</code>	Affiche les 5 derniers commits
<code>git log --author='Nom'</code>	Filtre par auteur
<code>git log --since='2024-01-01'</code>	Commits depuis une date

5.7 Navigation dans l'Historique

Commande	Description
<code>git checkout <commit></code>	Se déplace sur un commit spécifique (mode détaché)
<code>git checkout <branche></code>	Change de branche
<code>git checkout -- <fichier></code>	Restaure un fichier depuis le dernier commit
<code>git switch <branche></code>	Change de branche (commande moderne)
<code>git restore <fichier></code>	Restaure un fichier (annule les modifications)
<code>git restore --staged <fichier></code>	Retire un fichier de la zone de staging

5.8 Annulation et Réinitialisation

Commande	Description
<code>git reset HEAD~1</code>	Annule le dernier commit (garde les modifications en local)
<code>git reset --soft HEAD~1</code>	Annule le commit, garde les changements staged
<code>git reset --hard HEAD~1</code>	Annule le commit et supprime toutes les modifications
<code>git revert <commit></code>	Crée un nouveau commit qui annule un commit précédent

Commande	Description
<code>git clean -fd</code>	Supprime les fichiers non suivis (non stagés)

Différence entre reset et revert :

- `git reset` : modifie l'historique (dangereux si partagé)
- `git revert` : crée un commit d'annulation (sûr pour les branches partagées)

6. Branches

Les branches sont l'une des fonctionnalités les plus puissantes de Git. Elles permettent de travailler sur des fonctionnalités ou des correctifs de manière isolée, sans affecter le code principal.

6.1 Créer et Lister

Commande	Description
<code>git branch</code>	Liste toutes les branches locales (* = branche courante)
<code>git branch -a</code>	Liste toutes les branches (locales + distantes)
<code>git branch <nom></code>	Crée une nouvelle branche
<code>git branch -d <nom></code>	Supprime une branche (sécurisé)
<code>git branch -D <nom></code>	Force la suppression d'une branche
<code>git branch -m <ancien> <nouveau></code>	Renomme une branche

6.2 Switcher entre les Branches

Commande	Description
<code>git switch <branche></code>	Bascule sur une branche existante (commande moderne)
<code>git switch -c <branche></code>	Crée et bascule sur une nouvelle branche
<code>git checkout <branche></code>	Bascule sur une branche (commande historique)
<code>git checkout -b <branche></code>	Crée et bascule sur une nouvelle branche

6.3 Fusionner des Branches (Merge)

La fusion intègre les modifications d'une branche dans une autre.

```
# Depuis la branche cible (ex: main)
git switch main
git merge feature-login
```

Commande	Description
<code>git merge <branche></code>	Fusionne la branche spécifiée dans la branche courante
<code>git merge --no-ff <branche></code>	Force la création d'un commit de fusion
<code>git merge --abort</code>	Annule une fusion en cours
<code>git merge --squash <branche></code>	Fusionne en un seul commit

6.4 Types de Fusion

- Fast-Forward : Git déplace simplement le pointeur (pas de nouveau commit)
- 3-Way Merge : Git crée un commit de fusion avec deux parents
- Squash Merge : Combine tous les commits en un seul avant fusion

6.5 Résoudre un Conflit de Fusion

Un conflit survient quand deux branches modifient la même partie du même fichier. Git marque le fichier en conflit :

```
<<<<<<< HEAD
Contenu de la branche courante
=====
Contenu de la branche à fusionner
>>>>>>> feature-branch
```

Étapes pour résoudre un conflit :

1. Ouvrir le fichier en conflit et repérer les marqueurs
2. Choisir le contenu à conserver (ou combiner les deux)
3. Supprimer les marqueurs de conflit (<<<<<<<, =====, >>>>>>>)
4. Ajouter le fichier résolu : `git add <fichier>`
5. Finaliser avec : `git commit`

6.6 Rebase

`git rebase` réapplique les commits d'une branche sur une autre base, produisant un historique plus linéaire.

```
# Rebaser la branche feature sur main
git switch feature
git rebase main
```

Règle d'or : ne jamais rebaser une branche déjà partagée avec d'autres développeurs.

6.7 Stash (Remisage)

`git stash` permet de mettre de côté des modifications non committées pour travailler sur autre chose.

Commande	Description
<code>git stash -m 'message'</code>	Sauvegarde les modifications non committées
<code>git stash list</code>	Liste toutes les sauvegardes
<code>git stash pop</code>	Restaure et supprime la dernière sauvegarde
<code>git stash apply stash@{0}</code>	Restaure sans supprimer la sauvegarde
<code>git stash drop stash@{0}</code>	Supprime une sauvegarde spécifique
<code>git stash clear</code>	Supprime toutes les sauvegardes

7. Dépôt Distant

Un dépôt distant (remote) est une version de votre projet hébergée sur un serveur. GitHub, GitLab et Bitbucket sont les plateformes les plus populaires.

7.1 README et Initialisation

Le fichier README.md est la carte de visite de votre projet. Il doit contenir :

- Le nom et la description du projet
- Les instructions d'installation
- Les exemples d'utilisation
- Les informations de contribution
- La licence du projet

7.2 Gérer les Dépôts Distants

Commande	Description
<code>git remote add origin <url></code>	Associe un dépôt distant au projet local
<code>git remote -v</code>	Liste les dépôts distants avec leurs URLs
<code>git remote remove <nom></code>	Supprime un dépôt distant
<code>git remote rename <ancien> <nouveau></code>	Renomme un dépôt distant
<code>git remote set-url origin <url></code>	Modifie l'URL d'un dépôt distant

7.3 Push (Envoyer des Modifications)

Commande	Description
<code>git push origin <branche></code>	Envoie la branche vers le dépôt distant
<code>git push -u origin main</code>	Envoie et configure le suivi de la branche
<code>git push --all</code>	Envoie toutes les branches
<code>git push --tags</code>	Envoie les tags
<code>git push origin --delete <branche></code>	Supprime une branche distante

7.4 Fetch et Pull (Récupérer des Modifications)

Commande	Description
<code>git fetch origin</code>	Récupère les modifications distantes SANS fusionner
<code>git fetch --all</code>	Récupère depuis tous les dépôts distants
<code>git pull origin <branche></code>	Récupère ET fusionne les modifications

Commande	Description
<code>git pull --rebase</code>	Récupère et rebase au lieu de merger

Différence entre fetch et pull :

- git fetch : télécharge les données, vous décidez quand fusionner
- git pull : fetch + merge automatique (= git fetch + git merge)

7.5 Clone

Commande	Description
<code>git clone <url></code>	Clone le dépôt complet avec tout l'historique
<code>git clone <url> <dossier></code>	Clone dans un dossier spécifié
<code>git clone --depth=1 <url></code>	Clone superficiel (dernier commit uniquement)
<code>git clone -b <branche> <url></code>	Clone une branche spécifique

7.6 Collaborateurs

Sur GitHub, pour ajouter des collaborateurs à un dépôt :

6. Aller dans Settings > Collaborators du dépôt
7. Cliquer sur 'Add people'
8. Saisir le nom d'utilisateur GitHub ou l'email
9. Choisir le niveau d'accès (Read / Write / Admin)

7.7 Pull Request (PR)

Une Pull Request est une demande de fusion de votre branche dans une autre. C'est le mécanisme central de collaboration sur GitHub.

Flux de travail typique :

10. Forker ou cloner le dépôt
11. Créer une branche dédiée : `git switch -c feature/ma-fonctionnalite`
12. Faire les modifications et commiter
13. Pusher la branche : `git push origin feature/ma-fonctionnalite`
14. Créer la Pull Request sur GitHub avec description et reviewers
15. Après révision et approbation, fusionner la PR

7.8 Tags

Commande	Description
<code>git tag v1.0.0</code>	Crée un tag léger
<code>git tag -a v1.0.0 -m 'message'</code>	Crée un tag annoté (recommandé)

Commande	Description
<code>git tag</code>	Liste tous les tags
<code>git push origin v1.0.0</code>	Envoie un tag vers le distant
<code>git push origin --tags</code>	Envoie tous les tags

8. Références

8.1 Documentation Officielle

- Documentation officielle Git : <https://git-scm.com/doc>
- Pro Git (livre gratuit en ligne) : <https://git-scm.com/book/fr/v2>
- Documentation GitHub : <https://docs.github.com>
- Documentation GitLab : <https://docs.gitlab.com>

8.2 Ressources d'Apprentissage

- Learn Git Branching (interactif) : https://learngitbranching.js.org/?locale=fr_FR
- GitHub Skills : <https://skills.github.com>
- Atlassian Git Tutorials : <https://www.atlassian.com/fr/git/tutorials>
- Oh My Git! (jeu d'apprentissage) : <https://ohmygit.org>

8.3 Outils

- Git GUI officiel : <https://git-scm.com/downloads/guis>
- GitKraken : <https://www.gitkraken.com>
- Sourcetree : <https://www.sourcetreeapp.com>
- VS Code Git Integration : <https://code.visualstudio.com/docs/editor/versioncontrol>

8.4 Aide-mémoire

- GitHub Git Cheat Sheet : <https://education.github.com/git-cheat-sheet-education.pdf>
- Atlassian Git Cheat Sheet : <https://www.atlassian.com/git/tutorials/atlassian-git-cheatsheet>

9. Démonstration Pratique (Travaux Pratiques)

Ce TP illustre concrètement les concepts Git vus dans ce guide, en utilisant un simple fichier texte. Il a été réalisé sous Windows avec Git Bash.

9.1 Mise en Place de l'Environnement

```
# Naviguer vers le dossier de travail
cd Desktop/AWS/

# Créer le répertoire de travail du projet
mkdir projetGIT2
cd projetGIT2

# Créer un fichier de test
vim note.txt
# (Saisir : bonjour contenu note, puis sauvegarder)

# Vérifier le contenu
cat note.txt
# => bonjour contenu note
```

9.2 Initialisation du Dépôt Git

```
# Initialiser Git dans le répertoire
git init
# => Initialized empty Git repository in .../projetGIT2/.git/

# Vérifier la présence du dossier .git
ls -a
# => ./ ../ .git/ note.txt

# Vérifier le statut du fichier (non suivi = ??)
git status -s
# => ?? note.txt
```

9.3 Premier Commit

```
# Ajouter le fichier à la zone de staging
git add .

# Vérifier que le fichier est staged (A = Added)
git status -s
# => A  note.txt

# Créer le premier commit
git commit -m "premier commit"
# => [master (root-commit) 336e6ff] premier commit
# 1 file changed, 1 insertion(+)
```

9.4 Modification et Deuxième Commit


```
# Ajouter une deuxième ligne au fichier
echo "deuxieme ligne ajoutée" >> note.txt

# Vérifier les modifications (M = Modified, en rouge = non staged)
git status -s
# => M note.txt

# Voir le détail des modifications (+ = ajouté, - = supprimé)
git diff
# => @@ -1 +1,2 @@
#      bonjour contenu note
#      +deuxieme ligne ajoutée

# Ajouter et commiter
git add .
git commit -m "deuxieme commit"
# => [master 0b922b8] deuxieme commit

# Vérifier l'historique
git log --oneline
# => 0b922b8 deuxieme commit
# => 336e6ff premier commit
```

9.5 Annulation de Modification

```
# Ajouter une ligne de test
echo "Test annulation de modification" >> note.txt

# Vérifier (M = modification non commitée)
git status -s
# => M note.txt

# Annuler la modification (restaurer depuis le dernier commit)
git restore note.txt

# Vérification : la ligne de test a disparu
cat note.txt
# => bonjour contenu note
# => deuxieme ligne ajoutée
```

9.6 Travail avec les Branches

```
# Lister les branches (seule master existe)
git branch
# => * master

# Créer la branche 'main'
git branch main

# Basculer sur la branche main
git switch main
# => Switched to branch 'main'

# Modifier note.txt depuis la branche main
echo "Fichier note modifier a partir de main" >> note.txt
```

```
# Commiter depuis main
git add .
git commit -m "commit modification a partir de main"

# Retourner sur master et faire une modification différente
git checkout master
echo "Fichier note modifier a partir de master" >> note.txt
git add .
git commit -m "commit modification a partir de master"
```

9.7 Fusion et Résolution de Conflit

```
# Tenter de fusionner main dans master
git merge main
# => CONFLICT (content): Merge conflict in note.txt
# => Automatic merge failed; fix conflicts and then commit the result.

# Voir le conflit dans le fichier
cat note.txt
# => bonjour contenu note
# => deuxieme ligne ajoutée
# => <<<<<<< HEAD
# => Fichier note modifier a partir de master
# => =====
# => Fichier note modifier a partir de main
# => >>>>>>> main

# Éditer le fichier pour conserver les deux lignes
vim note.txt
# (Supprimer les marqueurs, garder les deux lignes)

# Résoudre le conflit
git add .
git commit -m "conflit resolu en fusionnant les 2"

# Vérifier que la fusion est complète
git merge main
# => Already up to date.
```

9.8 Synchronisation des Branches

```
# Synchroniser main avec les changements de master
git switch main
git merge master
# => Fast-forward
# => note.txt | 1 +

# Les deux branches ont maintenant le même contenu
cat note.txt
# => bonjour contenu note
# => deuxieme ligne ajoutée
# => Fichier note modifier a partir de master
# => Fichier note modifier a partir de main
```

9.9 Utilisation du Stash

```
# Faire une modification sans la commiter
echo "Creation de sauvegarde avec stash from main" >> note.txt

# Sauvegarder avec stash
git stash -m "premier sauvegarde"
# => Saved working directory and index state On main: premier sauvegarde

# Lister les sauvegardes
git stash list
# => stash@{0}: On main: premier sauvegarde

# La modification a disparu temporairement
cat note.txt    # ne montre pas la ligne ajoutée

# Restaurer la sauvegarde (et la supprimer de la liste)
git stash pop
# => Dropped refs/stash@{0}

# La modification est revenue
git status -s
# => M note.txt

# Annuler la modification non commitée
git restore note.txt
```

9.10 Connexion à GitHub

```
# Créer le dépôt sur GitHub (via l'interface web)
# Puis associer le dépôt distant
git remote add origin https://github.com/dvddy03/projetGIT2.git

# Vérifier les remotes configurés
git remote -v

# Pousser la branche main vers GitHub
git push -u origin main
# => Writing objects: 100% (15/15), 1.16 KiB | 397.00 KiB/s, done.
# => * [new branch]      main -> main
# => branch 'main' set up to track 'origin/main'.
```

Le dépôt est maintenant accessible publiquement sur GitHub avec l'intégralité de l'historique des commits.

9.11 Récapitulatif des Commandes du TP

Commande	Description
<code>git init</code>	Initialiser le dépôt
<code>git status -s</code>	Vérifier le statut en mode court
<code>git add .</code>	Ajouter tous les fichiers au staging
<code>git commit -m '...'</code>	Créer un commit
<code>git diff</code>	Voir les modifications non stagées
<code>git log</code>	Voir l'historique des commits

Commande	Description
<code>git restore <fichier></code>	Annuler des modifications locales
<code>git branch <nom></code>	Créer une branche
<code>git switch <branche></code>	Changer de branche
<code>git merge <branche></code>	Fusionner une branche
<code>git stash -m '...'</code>	Sauvegarder les modifications non commitées
<code>git stash pop</code>	Restaurer la dernière sauvegarde
<code>git remote add origin</code>	Associer un dépôt distant
<code>git push -u origin main</code>	Pousser vers GitHub

Fin du Guide — Bonne pratique avec Git !

papalioune03@gmail.com